分　数:	
评卷人:	

华中科技大学

本科生“智能控制”课程报告

题　目： Arena Challenge 智能车导航控制策略设计

选题方向:

✓ IC Challenge

☐ 自选题目

学　　号 _____ _____

姓　　名 _____

专　　业 人工智能

课程指导教师 _____

院（系、所） 人工智能与自动化学院

2026 年 5 月 30 日

目　　录

摘要 1
1　引言 2
2　方法或算法 3
2.1　总体框架 3
2.2　地图处理与障碍物膨胀 3
2.3　八邻域 A* 路径规划 5
2.4　路径后处理与控制器 5
2.5　安全修正与动态避障 6
3　软件结构和软件实现方法 7
3.1　文件组织与提交入口 7
3.2　类结构与环境调用关系 8
3.3　单步决策的软件实现链路 8
3.4　配置读取与状态缓存 9
3.5　本章小结 9
4　数据描述 10
5　实验结果 11
6　优化探索与消融实验 16
6.1　Exp3/Exp4 残差强化学习尝试 17
6.2　Exp5/Exp6 车体膨胀尝试 19
7　总结 20
7.1　工作总结 20
7.2　算法创新性 20
7.3　后续改进方向 20
8　运行与复现指导 22
8.1　环境与文件 22
8.2　复现主实验 23
8.3　复现探索实验 24
8.4　训练记录与随机种子 25
参考文献 27

基于 A* 与 PD 的 Arena Challenge 智能车导航控制策略

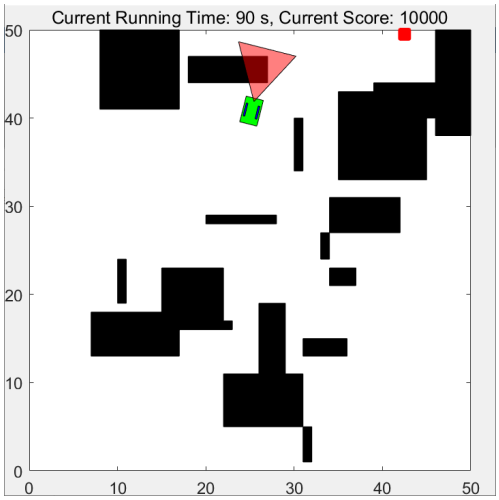
摘要：本课程设计面向 **Arena Challenge 智能车导航任务**，目标是在栅格地图中控制一辆小车从起点到达终点，并尽量减少**碰撞**、**超时和绕行**带来的扣分。我的最终实现是把问题拆成**“建图–规划–跟踪–安全修正”四层**：先根据全局地图或雷达扫描构建可规划地图，并通过**障碍物膨胀**给车体宽度留出安全裕度；再用**八邻域 A* 搜索**生成从当前位置到目标区域的路径；随后对贴墙路径做**几何偏置**，并提取路径拐点；最后根据路径形状分两种开法：路比较顺的时候让车稳着往前走，快到转弯处再用**误差反馈**把车头拉回规划路线，同时加入**前向碰撞预测**和**动态小车让行逻辑**。实验表明，在 **8 张固定地图**、**globalview=1** 的设置下，无 Smart-Agent 时小车**全部到达且得分均为 10000 分**；加入 **10 个 Smart-Agent** 后仍全部到达，在固定随机种子下**平均得分 9523.0**；加入 **20 个 Smart-Agent** 后也全部到达，**平均得分 9422.5**。同时在整个实验的过程中，我又分别尝试了 **Smart-Agent 速度预测**、**DWA 局部采样**、**车体 footprint 膨胀**以及 **A*+PD 残差 TD3 训练**。对比之后发现，当前这套 **A* 规划加误差反馈修正的分层方案**在**到达率**、**平均得分和运行稳定性**上综合效果最好，因此作为**最终提交策略**。

关键词：Arena Challenge；A* 路径规划；地图膨胀；动态避障；残差强化学习

1 引言

Arena Challenge 的任务看起来只是从起点到终点的寻路，但实际运行时并不是一个简单的静态最短路问题。小车的状态包括位置和朝向，控制量只有前进速度 u 和角速度 v，它不能像栅格点一样横向移动或瞬间改变朝向；同时，传感器信息可能是局部的，地图中还可能出现动态的 Smart-Agent。因此，如果只在原始地图上找一条最短折线，再让小车机械地追踪路径点，常见问题是贴墙、直角弯过冲、窄通道中振荡，或者在动态小车附近发生碰撞扣分。这类问题本质上同时涉及移动机器人导航中的路径规划、路径跟踪和动态避障几个层次 [4]。

我在设计策略时的基本动机是：先把“能不能安全走”做得保守一些，再在安全基础上尽量提高速度。规划层通过地图膨胀让路径远离障碍物；控制层根据直道和弯道切换不同控制方式；安全层在指令输出前做短时碰撞预测和动态障碍修正。这个方法的优点是每一部分都比较可解释，调试时也能看出问题来自规划、跟踪还是避障。



(a) Arena Challenge 仿真场景

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} u\cos\theta \\ u\sin\theta \\ v \end{bmatrix}$$

(b) 小车运动模型

图 1-1 课程挑战任务中的环境和车辆运动模型

2 方法或算法

2.1 总体框架

从总体上看，我设计的策略可以理解为一个分层闭环导航控制器。每一个控制周期中，小车先把当前位置、目标点、静态障碍和雷达范围内的动态目标统一整理成可用于规划的环境表示；然后在这个表示上寻找一条到终点附近的全局路径；接着根据路径形状对贴墙段和转弯段做修正；最后在真正输出速度之前，再进行一次安全检查，避免因为短时碰撞风险而执行过于激进的动作。

这里最重要的不是把每一种雷达监测到的情况都写成独立算法，而是保持同一套主逻辑：地图信息完整时，规划层可以直接利用全局占据图；地图信息不完整时，则把历史扫描结果逐步融合成局部地图。对于 Smart-Agent 这类会移动的障碍物，我没有把它们永久当作墙体写入地图，而是把它们作为短时风险，在控制输出前参与减速和让行判断。这样处理可以避免临时动态目标把本来可走的通道“堵死”。

整体流程如图 2-1 所示。它的核心思想可以概括为：先用地图膨胀和 A* 解决“从哪里走”的问题，再用路径形状修正和误差反馈解决“怎么开得稳”的问题，最后用碰撞预测和动态让行解决“当前这一步能不能安全执行”的问题。

2.2 地图处理与障碍物膨胀

地图处理的目标是把不同来源的感知信息统一成一张可规划的占据栅格。若环境信息完整，算法可以直接根据全局地图区分自由区域和障碍物区域；若只能看到局部雷达范围，则需要把当前扫描结果与历史观测结果融合，逐步补全小车已经探索过的区域。这样做的好处是，规划层始终面对同一种地图表达，而不需要因为能获得的地图信息的量不同而重新设计路径搜索方法。

考虑到小车本身有长宽，不能只把车看成一个没有面积的点。如果路径恰好贴着墙走，栅格中心看似没有撞上障碍物，但车体边角仍可能发生剐蹭。因此在规划前，我对静态障碍物做 3×3 膨胀，相当于在障碍物周围留出一圈安全缓冲区。这个处理会牺牲一部分狭窄空间，但换来的是更稳定的通行安全性，尤其是在转弯和贴墙长通道中效果比较明显。

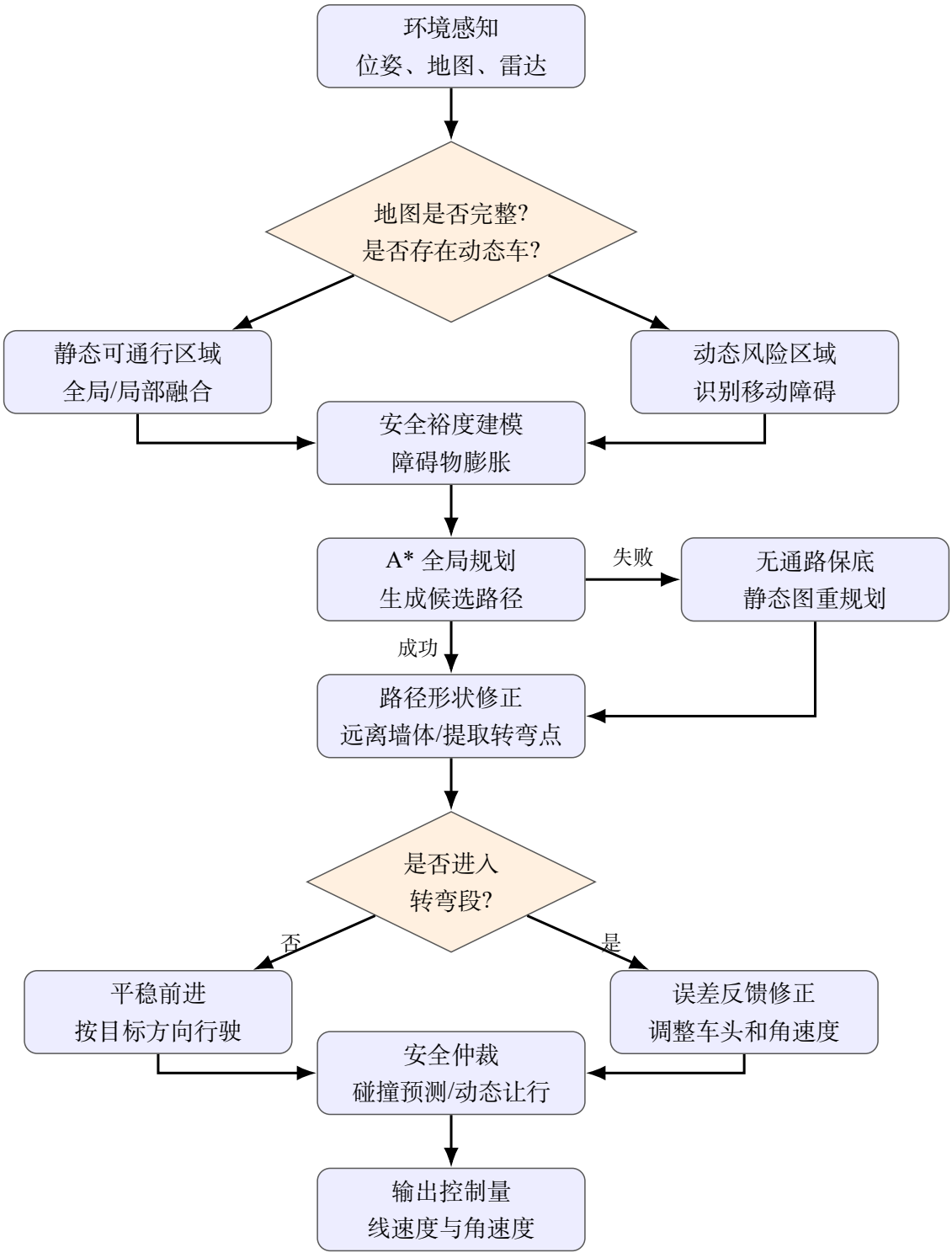


图 2-1　分层导航控制算法流程图

2.3 八邻域 A* 路径规划

A* 负责在膨胀后的栅格地图上寻找一条从当前位置到目标区域的候选路径[1]。搜索过程中，每个节点保存当前位置、累计代价、启发式代价以及父节点信息，可写成

$$[x,\ y,\ F,\ G,\ parent_x,\ parent_y], \tag{2-1}$$

其中 G 是从起点到当前节点的累计代价，H 是从当前节点到目标点的启发式估计，$F = G + H$。节点扩展采用八邻域形式，上下左右移动代价为 10，对角线移动代价为 14，用来近似斜向距离。启发函数采用曼哈顿距离：

$$H = 10\left(|x - x_{\text{goal}}| + |y - y_{\text{goal}}|\right). \tag{2-2}$$

这个启发函数不需要开方，计算量较小；对于 50×50 的地图规模，搜索效率和路径质量是可以接受的。

如果当前膨胀地图上暂时没有通路，规划层会采用保底处理：优先回到只考虑静态墙体的地图上重新规划，避免把临时出现的动态小车误认为永久障碍。这样可以减少因为局部遮挡或短时会车导致的路径中断。

2.4 路径后处理与控制器

原始 A* 路径是离散折线，容易在贴墙长段和直角弯处给控制器造成压力。因此在得到路径后，我先根据路径点与障碍物的相对位置做一个轻微的几何偏置：如果一段路径长期靠墙，就把这一段向远离墙的方向平移 0.5 个栅格，让小车更接近通道中线。随后再根据相邻路径段的方向变化提取转弯点，为后面的控制切换提供依据。

控制部分采用“顺路前进 + 转弯修正”的方式。路径比较平顺时，小车主要根据目标路径段方向调整车头，角度差很小时给较高线速度，角度差较大时主动降速并增加角速度修正。这一思路和车辆运动规划与控制中常见的“规划路径 + 局部跟踪控制”结构比较接近[4]。接近转弯点时，则构造横向误差

$$e = \sin(\Delta\theta) \cdot d, \tag{2-3}$$

并使用类似 PD 的反馈形式

$$u_\omega = K_p e + K_d(e - e_{\text{last}}), \tag{2-4}$$

其中，e 表示当前时刻用于修正转弯的横向误差；$\Delta\theta$ 表示车头朝向与目标路径段方向之间的夹角，角度越大说明车头偏离规划方向越明显；d 表示小车当前位置到目标路径点或转弯参考点的距离，用来反映“偏得有多远”；u_ω 表示最终叠加到角速度上的反馈修正量。K_p 和 K_d 分别是比例项和差分项的权重，本策略中取 $K_p = 650, K_d = 1000$；e_{last} 是上一控制周期的横向误差，因此 $e - e_{\text{last}}$ 可以近似表示误差变化趋势。这里采用的是 PD 误差反馈控制。这样设计后，控制器不只看车头角度，也会同时考虑车离路径还有多远，在弯道处比单纯追角度更稳一些。

2.5 安全修正与动态避障

安全修正是输出前的最后一道保护。算法根据候选线速度和角速度预测 0.3 s 后的小车中心和朝向，再由车体长宽计算四个角点、四条边中点和中心点共 9 个采样点。如果这些点越界或落在障碍物栅格上，就取消这次危险转向，避免因为短时动作过大而撞墙。

对于 Smart-Agent，策略没有把动态小车永久写入地图，因为它们会移动，直接当成静态墙体会造成虚假堵塞。我的处理方式是把动态目标转换到车体前向坐标系，判断其前向距离和横向偏移；如果动态小车出现在前方较近区域，就降低线速度，并根据其在左侧还是右侧给一个小的避让角速度。这个方法属于启发式局部避障，优点是简单直接，也比较适合放在输出前作为安全补丁。

3 软件结构和软件实现方法

本章主要说明策略在程序层面是如何组织和落地的，整体上，软件实现可以分成四个部分：第一节说明文件组织和提交入口；第二节说明 Policy 类与仿真环境之间的调用关系；第三节沿着 action(observation) 展开一次控制周期内部做了什么；第四节说明配置读取和状态缓存方式。这样安排的目的是把“软件结构”和“算法实现”对应起来。

3.1 文件组织与提交入口

本次程序以 MATLAB classdef 形式实现。最终提交入口是 Policy.m，它本身保持很短，只负责定义评测器要实例化的 Policy 类，并继承真正包含策略逻辑的 Policy_AStar_PD_Optimized。这样做的好处是入口文件清楚，主策略中的建图、规划、控制和安全修正逻辑也能集中维护。

表 3-1 主方法相关文件与函数职责划分

类型	名称	职责
文件	Policy.m	评测入口文件，提供评测器需要实例化的 Policy 类。
文件	Policy_AStar_PD_Optimized.m	主策略文件，保存建图、规划、路径后处理、控制和安全修正的主体逻辑。
成员函数	action(observation)	每个仿真步的决策入口，读取 observation 并返回控制量 $[u, v]$。
成员函数	A_star	八邻域 A* 搜索函数，负责在处理后的栅格地图上生成离散路径。
成员函数	add_bias / find_twist	对离散路径做几何偏置，并提取转弯点供控制器切换使用。
成员函数	easy_move / pd_move	分别处理顺路前进和转弯附近的误差反馈修正。
成员函数	predict_collide	在动作输出前做短时碰撞预测，过滤明显危险的转向。

3.2 类结构与环境调用关系

从软件结构看，仿真环境和策略之间是一个标准闭环：环境提供 observation，策略根据 observation 输出二维控制量 $[u, v]$，环境再根据动作推进一步并返回新的 observation。策略类本身不直接修改环境得分，也不跳过环境的动力学约束，而是只负责决定当前控制周期应该给出怎样的线速度和角速度。

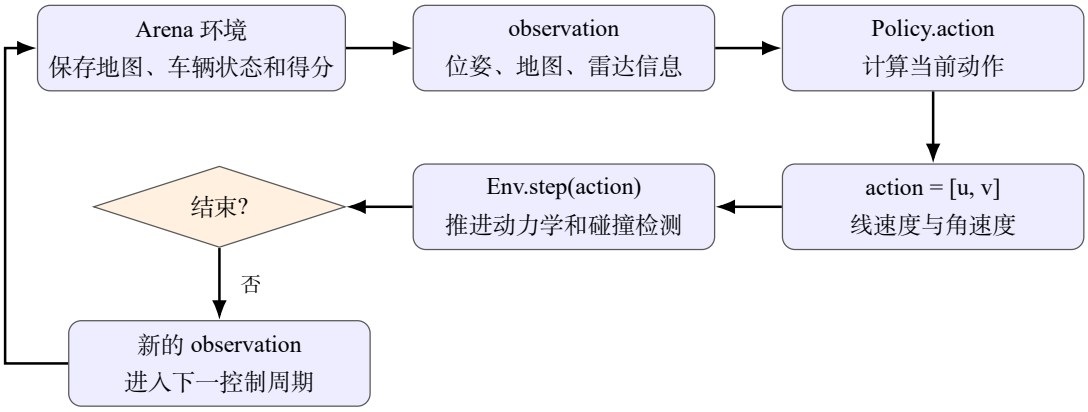


图 3-1 策略类与仿真环境的闭环调用关系

3.3 单步决策的软件实现链路

action(observation) 是策略实现的核心。一次调用并不是简单地查表输出动作，而是按“感知整理–地图更新–路径规划–路径修正–控制输出–安全仲裁”的顺序完成。这个链路与第 2 章的方法对应，但本节更强调程序中的执行先后和模块边界。

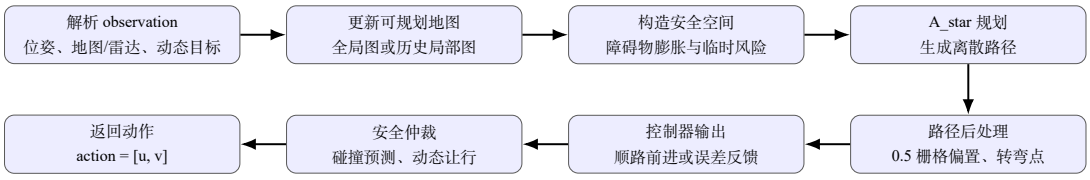


图 3-2 action 函数内部的软件执行链路

在实现上，我把最容易出问题的部分放在链路后半段处理。比如 A* 给出的路径如果贴墙，先由路径后处理做几何偏置；控制器给出的角速度如果会让车体角点撞上障碍物，再由安全仲裁进行过滤。这样每一层都只处理自己最擅长的问题，整体调试也更清楚。

3.4 配置读取与状态缓存

sys.ini 是环境和策略共同依赖的配置入口，里面包含地图模式、起终点、速度饱和、仿真步长、是否显示窗口以及 Smart-Agent 数量等信息。策略在构造函数中读取这些配置，并保存地图尺寸、起终点、车体尺寸和控制饱和值。这样做可以避免每次 action 都重复解析配置文件，也能保证策略和当前仿真设置一致。

除了固定配置，策略内部还会缓存少量随时间更新的状态，例如历史局部地图、上一周期的路径、上一周期的横向误差以及动态目标的短时信息。这些缓存只用于提高连续决策的稳定性，不改变环境本身的状态。每次重新开始一轮评测时，需要重新实例化 Policy，否则旧地图或旧误差可能影响新地图的判断。

3.5 本章小结

从软件实现角度看，最终策略并不是把所有逻辑都堆在一个函数里，而是围绕 Policy.action 形成一个清楚的闭环：外层由 Arena 环境负责推进仿真，策略类只负责根据当前 observation 计算动作；内层由地图、规划、路径后处理、控制和安全仲裁几个模块顺序完成一次决策。这样的组织方式比较适合本课程设计：一方面便于向评测器提交，另一方面也方便在本地替换某个实验模块做对照。

本章只讨论软件结构和实现方法。具体如何启动 MATLAB、如何批量评测、随机种子如何设置，会在运行与复现指导这一章节单独说明，避免把复现步骤和程序结构混在一起。

4 数据描述

实验使用课程提供的 Arena Challenge 仿真环境。地图大小为 50×50，起点和终点由 sys.ini 中的 StartPos、EndPos 指定。小车控制量为 $[u, v]$，其中 u 为前向速度，v 为角速度；当前配置中 Agent.usat=1、Agent.vsat=0.5，仿真步长 st=0.3 s，最大仿真时间 tend=150 s。

测试地图包括 amap、a1map、a2map、bmap、b1map、b2map、cmap 和 c1map。为了保证实验可复现，批量实验中设置 Map.random=0、System.showViewer=0、System.record=0；主实验分别设置 smartAgent=0、10 和 20，用来观察动态小车数量增加后得分和到达稳定性的变化。随机种子统一使用 seedBase=20240527，各地图对应关系会在实验结果和运行复现章节中说明。

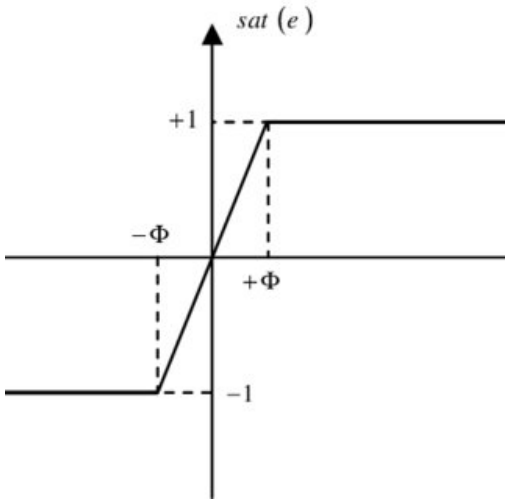


图 4-1　仿真环境中的控制量饱和机制

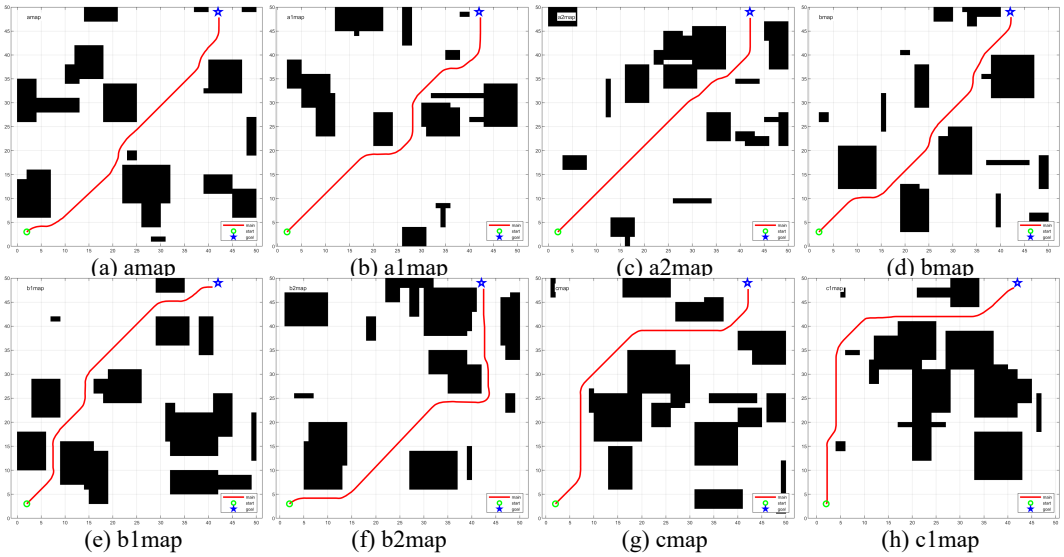
5 实验结果

实验分别测试 0、10、20 个 Smart-Agent 三种场景。三组实验都使用 8 张固定地图，且随机种子基准均为 seedBase=20240527；也就是说 amap 使用 20240527，a1map 使用 20240528，依次类推到 c1map 使用 20240534。评估指标包括最终得分 score、完成时间 time、仿真步数 steps 和是否到达 reached。其中 reached 表示最终位置与终点在 x、y 方向上均小于 1 个单位。

表 5-1 主实验设置

项目	设置
地图	amap、a1map、a2map、bmap、b1map、b2map、cmap、c1map，共 8 张固定地图。
Smart-Agent 数量	分别取 0、10、20，用来观察动态障碍数量增加后的影响。
随机种子	seedBase=20240527；8 张地图依次使用 20240527–20240534。
环境设置	globalview=1，Map.random=0，showViewer=0，record=0，仿真步长 st=0.3 s。
输出文件	bias05_policy_static.csv、bias05_policy_dynamic10.csv、bias05_policy_dynamic20.csv。

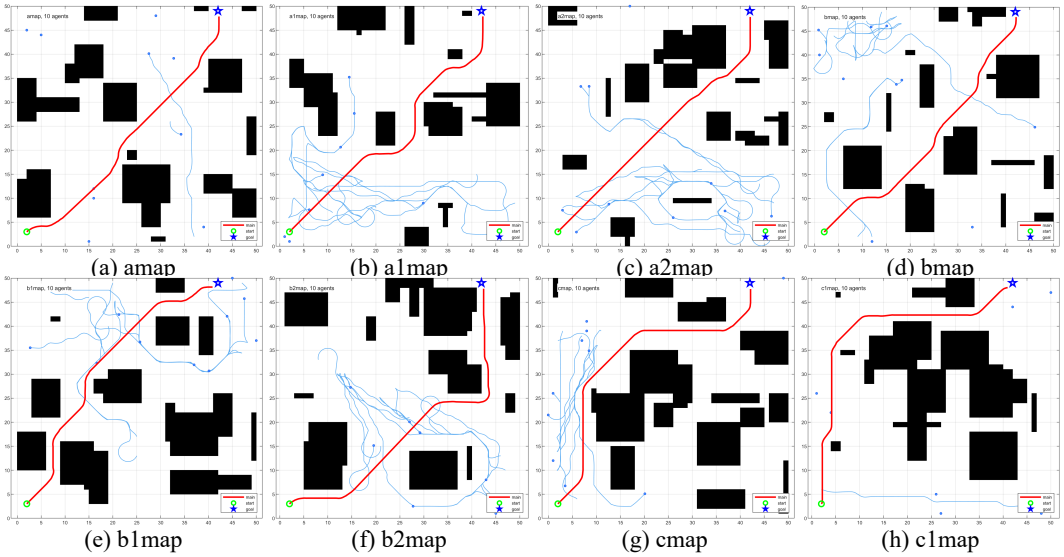
下面三张图给出每轮主实验的完整运行轨迹。每张小图对应一张地图，红线为主车实际行驶轨迹，浅蓝色细线为 Smart-Agent 在本轮仿真中走过的位置，蓝色小圆点表示 Smart-Agent 结束时的位置；黑色区域为障碍物，绿色圆点和蓝色星号分别表示主车起点和终点。这样画的目的不是只展示一条规划线，而是把动态障碍参与运行的过程也放进同一张图里。



(a) amap (b) a1map (c) a2map (d) bmap

(e) b1map (f) b2map (g) cmap (h) c1map

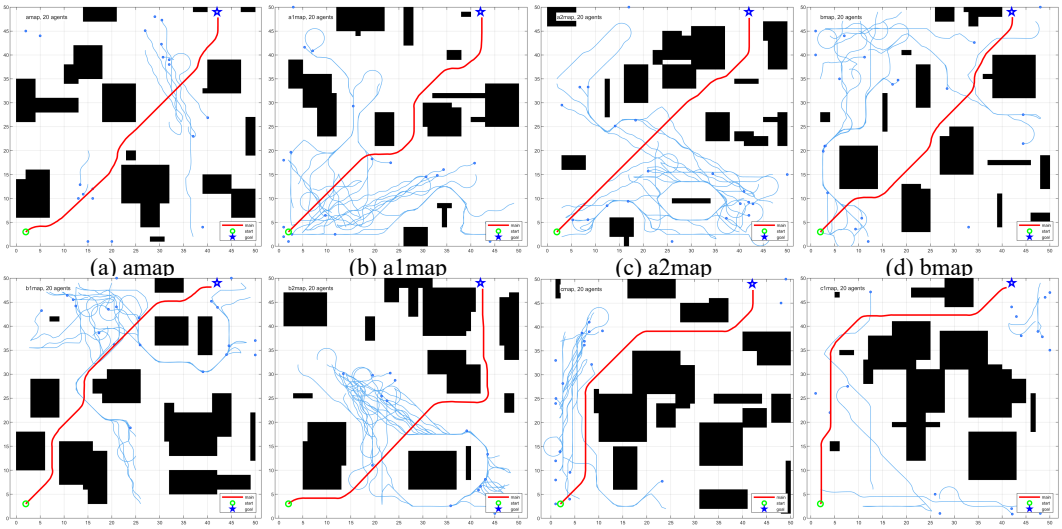
图 5-1　0 个 Smart-Agent 时 8 张地图的运行轨迹



(a) amap (b) a1map (c) a2map (d) bmap

(e) b1map (f) b2map (g) cmap (h) c1map

图 5-2　10 个 Smart-Agent 时 8 张地图的运行轨迹



(a) amap (b) a1map (c) a2map (d) bmap

(e) b1map (f) b2map (g) cmap (h) c1map

图 5-3　20 个 Smart-Agent 时 8 张地图的运行轨迹

表 5-2　无 Smart-Agent 时的评测结果

地图	得分	时间/s	步数	到达
amap	10000	66.6	222	是
a1map	10000	70.5	235	是
a2map	10000	64.8	216	是
bmap	10000	67.2	224	是
b1map	10000	69.3	231	是
b2map	10000	76.8	256	是
cmap	10000	74.4	248	是
c1map	10000	79.5	265	是

无动态小车时，8 张地图全部到达，得分均为 10000，平均完成时间为 71.14 s。最快的是 a2map，用时 64.8 s；最慢的是 c1map，用时 79.5 s。这说明在静态障碍环境下，地图膨胀、A* 规划和双模态控制器配合比较稳定，没有出现影响得分的碰撞或超时。

表 5-3　10 个 Smart-Agent 时的评测结果

地图	得分	时间/s	步数	到达
amap	9650	66.6	222	是
a1map	9597	70.5	235	是
a2map	9389	64.8	216	是
bmap	9749	67.2	224	是
b1map	9336	69.3	231	是
b2map	9577	76.8	256	是
cmap	9319	74.4	248	是
c1map	9567	78.6	262	是

加入 10 个 Smart-Agent 后，8 张地图仍全部到达，平均完成时间为 71.03 s，平均得分为 9523。相比静态场景，主要变化是动态会车造成了一部分碰撞或近距离干扰扣分，但路线本身没有被破坏，说明输出前的减速和让行逻辑能把动态干扰控制在可接受范围内。

表 5-4　20 个 Smart-Agent 时的评测结果

地图	得分	时间/s	步数	到达
amap	9577	66.6	222	是
a1map	9260	70.5	235	是
a2map	9197	64.8	216	是
bmap	9574	67.2	224	是
b1map	9395	69.3	231	是
b2map	9600	76.8	256	是
cmap	9590	74.4	248	是
c1map	9187	78.6	262	是

加入 20 个 Smart-Agent 后，8 张地图仍全部到达，平均完成时间为 71.03 s，平均得分为 9422.5。与 10 个 Smart-Agent 相比，动态目标更多，平均得分继续下降，但完成时间几乎没有变化，说明最终策略的主路线跟踪比较稳定，主要扣分来自

会车过程中的局部接触或过近干扰。最低分出现在 c1map 和 a2map 一类路线较长或会车点较集中的地图，这也提示后续优化不能只追求更激进的避让，还要避免把原本已经稳定的沿线行驶逻辑打乱。

表 5-5　三组主实验汇总

Smart-Agent 数量	平均得分	最低得分	平均时间/s	到达数
0	10000.0	10000	71.14	8/8
10	9523.0	9319	71.03	8/8
20	9422.5	9187	71.03	8/8

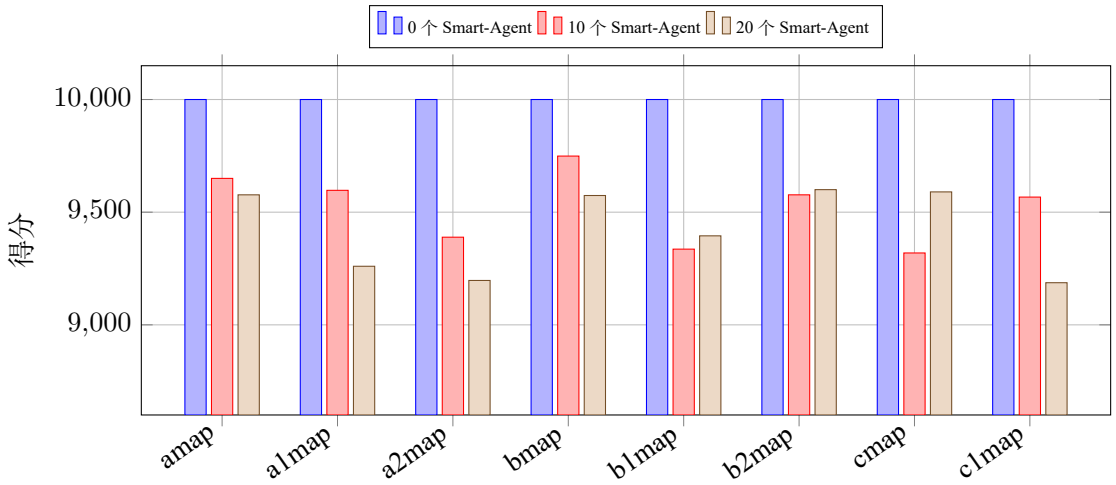


图 5-4　不同地图下的得分对比

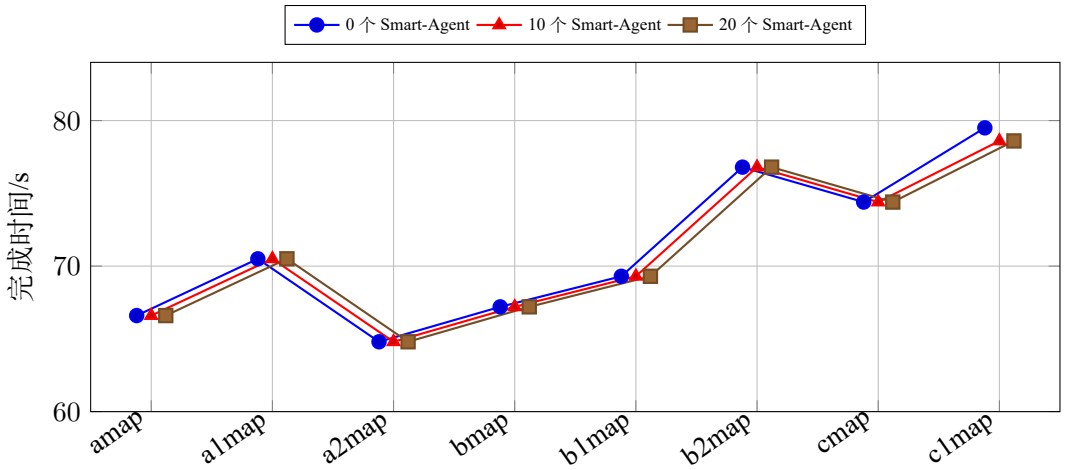


图 5-5　不同地图下的完成时间对比

6 优化探索与消融实验

当然，目前的方案并不是一开始就完整想出来的。前期我也做了很多实验和排错：有的想法从直觉上看更合理，例如让小车预测 Smart-Agent 的速度、用 DWA 做局部采样、把固定地图膨胀改成车体膨胀，或者用残差强化学习去补偿主控制器。经过多轮对比后，我最后确定使用当前这套 A* 规划加误差反馈修正的方案，因为它在平均得分、最低得分和到达稳定性上综合效果最好。下面把这些探索版本的具体做法和实验结果放出来，并和最终方案做对比。

表 6-1 优化探索的设计动机

实验策略	主要改动
Exp1 动态预测	对 scanMap 中的 Smart-Agent 做中心点匹配，估计速度，并预测约 1–2 s 后的位置；只在预测轨迹进入车体前向通道时降低速度或给出小角速度修正。这个方向参考了动态障碍避让中“速度–未来位置”风险判断的基本思想 [3]。
Exp2 DWA 混合	保留 A* 给出的全局路径和原控制器，在动态障碍靠近时，对当前速度附近的若干 $[u, v]$ 进行短时轨迹采样，用进度、朝向、清障距离和动态风险打分。该实验借鉴了 Dynamic Window Approach 在速度空间内采样并评价短时轨迹的思想 [2]。
Exp3/Exp4 残差 TD3	不替换原控制器，而是把主策略动作、路线误差、局部扫描和 Smart-Agent 预测风险组成状态，训练 actor 输出很小的 $[\Delta u, \Delta v]$ 残差。TD3 本身属于 actor–critic 类连续控制算法 [5]。
Exp5/Exp6 车体膨胀	不再固定把每个障碍物做 3×3 膨胀，而是根据主车 3×2 的 footprint 或车体半对角线生成配置空间，检查这种更物理一致的膨胀是否能减少贴墙碰撞。

为了保证实验可复现，所有完整对比实验都使用 8 张固定地图和同一组随机种子：seedBase=20240527，amap 到 c1map 依次使用 20240527–20240534。表 6-2 给出 20 个 Smart-Agent 时几个代表性探索方案的对比结果。残差 TD3 属于训练型探索，训练过程和固定地图评测的统计口径不同，因此单独在后文列出训练记录。

表 6-2　20 个 Smart-Agent 下的探索实验汇总

策略	平均得分	最低分	平均时间/s	到达数
最终方案	9423.8	9187	71.0	8/8
Exp1 动态预测	9195.1	8522	76.4	8/8
Exp2 DWA 混合	8734.6	5605	98.7	6/8

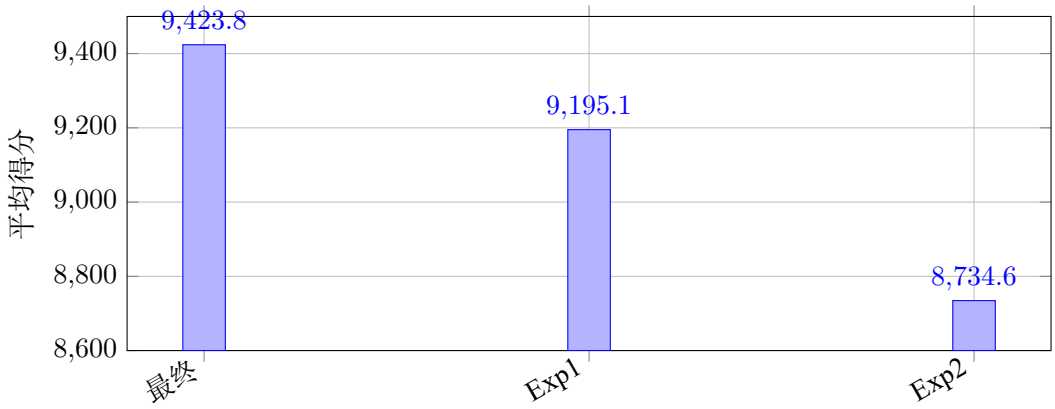


图 6-1　20 个 Smart-Agent 下代表性探索方案与最终方案的平均得分对比

从结果看，最终方案的平均得分和到达稳定性最好。Exp1 的 Smart-Agent 速度预测说明“提前看对方运动趋势”这个思路是有价值的，但预测误差会让主车过早减速，反而增加后续相遇的不确定性。Exp2 的 DWA 混合方法在局部避障上更主动，但短时评分函数容易过度关注眼前障碍，牺牲沿着全局路线前进的进度，因此 20 个 Smart-Agent 下有两张图没有到达。后续报告重点讨论动态预测、DWA、残差 TD3 和车体膨胀这几类探索，它们分别对应动态障碍处理、局部轨迹采样、学习型补偿和安全裕度建模四个方向。

6.1　Exp3/Exp4 残差强化学习尝试

在启发式方法之外，我还尝试过在当前主策略上加入端到端的残差强化学习。这里的“端到端”指训练过程端到端：环境状态经过 actor 后直接给出残差动作，并通过 TD3 的 critic 学习长期回报；但控制结构不是完全替换原来的 A* 规划和误差修正，而是保留主策略动作，只学习一个小幅补偿。这样设计的动机是，当前主策略已经能稳定到达，强化学习更适合学习在 Smart-Agent 干扰下何时稍微减速、加速或调整角速度。强化学习在机器人控制中常被用来学习难以手工整定的局部行

为 [7]，而 TD3 则是在连续动作 actor–critic 框架中通过双 critic、延迟策略更新等方式缓解过估计问题 [5]。

具体做法是：每一步先由 A*+PD 输出 $u_{\text{main}}, v_{\text{main}}$，再把车辆位姿、目标方向、waypoint 误差、主策略动作、局部障碍信息以及 8 维 Smart-Agent 预测特征组成 34 维状态。TD3 actor 输出归一化残差，经

$$\Delta u = 0.10 a_1, \qquad \Delta v = 0.08 a_2 \tag{6-1}$$

缩放后叠加到主策略动作上。其中 a_1 和 a_2 是 actor 输出的两个归一化动作分量，通常被限制在 $[-1, 1]$；Δu 表示叠加到主策略线速度上的残差，Δv 表示叠加到主策略角速度上的残差；0.10 和 0.08 是残差幅值系数，用来保证学习策略只做小幅修正，而不是直接推翻主控制器。最终执行动作可写为 $u = u_{\text{main}} + \Delta u,\ v = v_{\text{main}} + \Delta v$。为了避免学习策略破坏稳定性，我又加了两层限制：只有预测风险 gate 打开时才允许残差生效；执行前还会用 0.6 s 短视距安全检查把残差缩放为 $1, 0.5, 0.25$ 或直接退回主策略动作。

表 6-3　Exp3/Exp4 残差 TD3 训练阶段记录（探索实验）

统计窗口	平均得分	到达率	说明
最近 100 局	9261.4	98.0%	约第 500 局附近的训练状态
最近 200 局	9308.9	97.5%	大部分 episode 能完成，但仍有波动
最佳连续 30 局	9422.3	100.0%	第 269–298 局，已经接近最终方案
最终方案	9423.8	100.0%	固定 8 张地图评测结果

这个实验给我的结论比较直接：残差 TD3 能学到一些局部调整动作，但训练过程的波动比手工分层方案更大。后来检查代码链路时也发现，当前 replay buffer 中记录的是 actor 期望输出的残差，而 MATLAB 端实际执行的动作还会经过安全缩放和饱和处理；如果这两个动作不一致，critic 学到的 $Q(s, a)$ 就会带噪声。换句话说，强化学习方向是值得继续尝试的，但在本次课程设计的时间内，当前 A* 规划加误差反馈修正的分层方案更稳定、复现性更好，因此最终采用这一版。

6.2 Exp5/Exp6 车体膨胀尝试

在处理安全裕度时，我一开始想先尝试“小车膨胀”：不直接把地图障碍物统一膨胀，而是根据小车 footprint 构造配置空间。这个思路从几何上更自然，因为它等价于考虑车体真实占用范围；但后面的实验表明，在本题的窄通道地图里，它的综合效果反而没有当前的地图膨胀稳定。主车在仿真中的尺寸由 Agent 类设置为 $w = 3, ht = 2$，因此我写了两类实验：Exp5 使用多朝向 footprint 的并集做膨胀，Exp6 使用车体半对角线加少量栅格余量的圆盘膨胀。为了避免不可达地图导致 A* 长时间搜索，我还加入了连通性检查，如果膨胀后起点到终点断开，就回退到原 3×3 膨胀。

表 6-4　车体膨胀诊断实验

策略	地图	随机种子	得分	时间/s	到达
Exp5 多朝向 footprint	amap	20240527	9557	69.3	是
Exp5 多朝向 footprint	b1map	20240531	9704	150.3	否
Exp6 圆盘 footprint	b1map	20240531	9583	150.3	否

结果说明，车体膨胀这个想法在几何上更讲得通，但放到本题的窄通道地图里不一定更合适。Exp5 在 amap 上可以正常到达，但路线更保守，完成时间变长；在 b1map 上虽然诊断输出的分数看起来不低，但实际没有到达终点，并且车体已经偏离有效路径。Exp6 的圆盘膨胀更温和，但在 b1map、seed=20240531 下仍然超时未达。这个实验帮助我确认了一点：当前固定 3×3 膨胀虽然简单，却和后面的 A* 路径、贴墙偏置、误差反馈修正配合得更自然，综合效果最好。

综合这些实验，我的筛选标准不是看某个模块名字是否更高级，而是看 8 张固定地图上的平均得分、到达数和运行稳定性。动态预测、DWA、车体膨胀和残差 TD3 都提供了有用的调试信息；但真正放到同一评测设置下比较后，当前方案的综合效果最好，所以最终 Policy 使用这一套主干逻辑，其他实验代码保留为后续改进的依据。

7 总结

7.1 工作总结

本课程设计完成了一套面向 Arena Challenge 的智能车导航控制策略。整体上，我没有采用端到端黑箱控制，而是把问题拆成地图处理、路径规划、路径修正、运动控制和安全检查几个层次。这样做的好处是每一层都比较容易解释，也便于在出问题时定位原因。实验结果表明，该策略在 8 张静态地图上能够稳定满分，在加入 10 个和 20 个 Smart-Agent 后也能保持全部到达，说明当前方案的稳定性和复现性比较好。

7.2 算法创新性

这次策略的主要创新不在于单独使用某一个复杂模型，而在于把几个简单模块组合成了比较稳的闭环。第一，地图层用障碍物膨胀给车体预留安全裕度，避免 A* 只规划出贴墙可行、实际难以跟踪的路线。第二，规划后没有直接跟踪原始折线，而是根据障碍物相对位置做轻微路径偏移，并提取拐点，使车辆在贴墙长段和直角弯处更容易开得平顺。第三，控制层采用“顺路开过去”和“拐弯前纠偏”两种状态切换，既保持直线段速度，又在转弯附近用误差反馈压住偏航。第四，安全层在动作输出前再做短视距碰撞预测和动态让行，使 Smart-Agent 场景下的动作不会只依赖单次 A* 结果。

另外，最终方案并不是直接确定的。报告中保留了动态预测、DWA 混合、残差 TD3 和车体膨胀几类探索实验。它们分别尝试从动态障碍预测、局部轨迹采样、学习型残差补偿和几何安全裕度建模四个方向改进策略。通过同一组地图和随机种子对比后，当前 A* 规划加误差反馈修正的方案在平均得分、最低分和到达数上综合表现最好，因此最终采用这一版。

7.3 后续改进方向

当前方案仍然有继续提升的空间。首先，动态避障仍然偏启发式，Smart-Agent 的速度估计和轨迹匹配可以做得更稳定，否则提前预测有时会变成过早减速。其次，DWA 或 MPC 这类局部规划方法并不是不能用，但需要把“沿全局路径前进”的进度约束做得更强，避免局部评分只顾眼前避障。再次，残差强化学习方向值得

继续尝试，不过训练时必须保证 replay buffer 记录的动作、MATLAB 端实际执行的动作和奖励函数严格一致，否则 actor 学到的可能是扰动而不是补偿。最后，车体膨胀可以作为自适应安全裕度继续研究，但不能简单替换当前地图膨胀；更合适的做法可能是根据通道宽度、转弯半径和车体姿态动态调整裕度。

8 运行与复现指导

本章说明本课程设计的运行环境、文件组织、评测命令和随机种子设置。前文已经给出了算法设计和实验结果，本章主要补充实验复现过程，便于检查报告中的主实验、探索实验和残差 TD3 训练记录。复现时不需要重新整理工程，只需要解压提交的复现包，并按照本章给出的目录和命令运行即可。

8.1 环境与文件

实验主要在 Windows 环境下完成，MATLAB 版本需支持 classdef 类定义和 rng 随机数设置。最终策略和普通评测只依赖 MATLAB；残差 TD3 训练还需要 Python 环境、numpy、torch 以及 MATLAB Engine for Python。复现包按“代码、地图、结果、训练记录”四类整理：code/ 中放置全部可执行代码和配置文件，maps/ 中放置 8 张固定地图，results/ 中保留报告使用的 CSV 结果，astar_residual_td3_runs/ 中保存残差 TD3 的训练日志和权重。

运行 MATLAB 评测时应将 Current Folder 切换到 code/。这样做有两个原因：一是 MATLAB 的 classdef 类和同名文件需要位于当前搜索路径中；二是仿真环境在读取地图时会从当前目录出发访问 ../maps/。若在复现包根目录直接运行，可能出现找不到策略类或地图文件的问题。表 8-1 给出了主要文件的作用。

表 8-1　主要复现文件说明

文件 / 目录	说明
code/Policy.m	评测入口，继承 Policy_AStar_PD_Optimized。
code/Policy_AStar_PD_Optimized.m	最终策略的完整实现（建图、A*、路径修正、控制、安全仲裁）。
code/sys.ini	仿真配置文件（地图名、起终点、Smart-Agent 数量、步长等）。
code/main.m	单图交互式运行入口，可观察小车实时行驶。
code/eval_policy_class_maps.m	批量评测脚本，自动遍历 8 张地图并输出 CSV 结果。
code/Env.m, code/Agent.m, code/Map.m 等	仿真环境核心代码（由课程提供）。
code/Policy_Exp*.m	消融实验策略文件（Exp1 动态预测、Exp2 DWA 等）。
code/Policy_AStar_PD_Footprint*.m	车体膨胀实验策略（Exp5/Exp6）。
maps/	8 张固定地图；code/Map.m 通过 ../maps/ 读取。
results/bias05_policy_*.csv	主实验评测结果（0/10/20 个 Smart-Agent）。
results/exp20_*.csv	探索实验评测结果。
code/train_astar_residual_td3.py	残差 TD3 训练主脚本。
TD3 核心 Python 模块	Actor、Critic 和 ReplayBuffer 的实现。
astar_residual_td3_runs/	残差 TD3 训练记录（含配置、指标和日志）。

8.2　复现主实验

主实验验证最终策略在 0、10、20 个 Smart-Agent 三种场景下的表现。运行前将 MATLAB 的 Current Folder 切换到文件包的 code/ 目录，并确认 sys.ini 中 globalview=1。评测脚本会自动设置地图名、Smart-Agent 数量、showViewer=0 和 record=0，运行结束后会恢复原始配置。

```
eval_policy_class_maps('Policy', 0, [], 20240527, ...
   '../results/verify_static.csv')

eval_policy_class_maps('Policy', 10, [], 20240527, ...
   '../results/verify_dynamic10.csv')
```

```
eval_policy_class_maps('Policy', 20, [], 20240527, ...
   '../results/verify_dynamic20.csv')
```

三条命令分别对应静态场景、10 个 Smart-Agent 场景和 20 个 Smart-Agent 场景。输出 CSV 中记录每张地图的得分、完成时间、步数、到达情况和随机种子。运行完成后，可与 ../results/ 中已有的主实验 CSV 对比。

表 8-2　批量评测命令参数说明

参数	含义
'Policy'	要评测的策略类名。最终提交策略使用 Policy。
0/10/20	动态 Smart-Agent 数量。本文主实验分别使用 0、10、20 三档。
[]	地图列表参数，留空时使用默认 8 张地图：amap、a1map、a2map、bmap、b1map、b2map、cmap、c1map。
20240527	随机种子基准 seedBase。8 张地图依次使用 seedBase 到 seedBase+7（见表 8-3）。
输出 CSV 文件名	结果保存文件，记录每张地图的得分、时间、步数、到达情况和实际使用的随机种子。

8.3　复现探索实验

探索实验使用与主实验相同的批量评测框架，只需将策略类名替换为对应实验策略。下面两条命令分别复现 20 个 Smart-Agent 下的动态预测实验和 DWA 混合实验：

```
eval_policy_class_maps('Policy_Exp1_DynamicPrediction', ...
   20, [], 20240527, '../results/verify_exp1.csv')

eval_policy_class_maps('Policy_Exp2_DWAHybrid', ...
   20, [], 20240527, '../results/verify_exp2.csv')
```

车体膨胀实验属于诊断性实验，报告中主要记录 amap 和 b1map 的单图结果。复现时在 sys.ini 中指定地图和 Smart-Agent 数量，然后实例化相应策略。例如复现 Exp5 在 b1map 上的测试，可将 name 设为 'b1map'，将 smartAgent 设为 20，并执行：

```
rng(20240531, 'twister');
env = Env('sys.ini');
policy = Policy_AStar_PD_FootprintInflation();
```

```
observation = env.reset();
done = 0;
while ~done
   action = policy.action(observation);
   [observation, done, ~] = env.step(action);
end
disp(observation.score);
```

单图观察时可将 showViewer 设为 1；批量统计时建议将 showViewer 和 record 均设为 0。

8.4 训练记录与随机种子

残差 TD3 采用 Python 训练脚本与 MATLAB 仿真环境交互。训练记录保存在 astar_residual_td3_runs/formal_20260527_231101/ 中，其中 config.json 记录训练超参数，metrics.txt 记录逐 episode 的地图、Smart-Agent 数量、得分、步数和到达情况，train_stdout.log 记录训练输出。如果重新训练，可在文件包根目录运行：

```
python code/train_astar_residual_td3.py --codex-dir code --run-dir astar_residual_td3_runs/retrain_20240527
```

训练脚本会在 8 张地图和不同 Smart-Agent 数量之间采样 episode，并定期导出权重检查点。由于强化学习训练本身存在随机性，重新训练的逐 episode 曲线不要求逐行一致，报告中使用训练记录主要说明残差策略的探索过程和波动情况。

本文所有定量评测使用固定的随机种子基准 seedBase=20240527。批量评测脚本通过 rng(seedBase + i - 1, 'twister') 为第 i 张地图设置随机种子，因此 Smart-Agent 的初始位置和运动轨迹在同一种子下是确定性的。表 8-3 列出了 8 张地图与随机种子的对应关系。

表 8-3　批量评测中的地图与随机种子对应关系

地图	amap	a1map	a2map	bmap
随机种子	20240527	20240528	20240529	20240530
地图	b1map	b2map	cmap	c1map
随机种子	20240531	20240532	20240533	20240534

在相同的 MATLAB 版本和操作系统下，使用上述种子运行批量评测，静态场景（0 个 Smart-Agent）的结果应当完全一致；动态场景由于 Smart-Agent 的物理模

拟依赖浮点精度，不同硬件上可能出现极小的数值偏差，但得分差异通常不超过个位数，到达情况不受影响。

参考文献

[1] Hart P. E., Nilsson N. J., Raphael B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths[J]. IEEE Transactions on Systems Science and Cybernetics, 1968, 4(2): 100–107.

[2] Fox D., Burgard W., Thrun S. The Dynamic Window Approach to Collision Avoidance[J]. IEEE Robotics & Automation Magazine, 1997, 4(1): 23–33.

[3] Fiorini P., Shiller Z. Motion Planning in Dynamic Environments Using Velocity Obstacles[J]. The International Journal of Robotics Research, 1998, 17(7): 760–772.

[4] Paden B., Cap M., Yong S. Z., Yershov D., Frazzoli E. A Survey of Motion Planning and Control Techniques for Self-Driving Urban Vehicles[J]. IEEE Transactions on Intelligent Vehicles, 2016, 1(1): 33–55.

[5] Fujimoto S., van Hoof H., Meger D. Addressing Function Approximation Error in Actor-Critic Methods[C]//Proceedings of the 35th International Conference on Machine Learning. Proceedings of Machine Learning Research, 2018, 80: 1587–1596.

[6] Lillicrap T. P., Hunt J. J., Pritzel A., Heess N., Erez T., Tassa Y., Silver D., Wierstra D. Continuous Control with Deep Reinforcement Learning[EB/OL]. arXiv:1509.02971, 2015.

[7] Kober J., Bagnell J. A., Peters J. Reinforcement Learning in Robotics: A Survey[J]. The International Journal of Robotics Research, 2013, 32(11): 1238–1274.

本科生签字: